Can a Machine Learning Model Consistently Learn Profitable Trading Strategies in the Forex Market?

(Technical Appendix)

Ethan Pedersen
Computer Science Department
Brigham Young University
Provo, United States
ethanp55@byu.edu

Jacob W. Crandall
Computer Science Department
Brigham Young University
Provo, United States
crandall@cs.byu.edu

I. Code

We provide our code in the supplementary "code.zip" file. There is a readme.md file that describes the code layout. Our language of choice was Python 3.9.

II. Generators (Trained Rules)

We implemented fourteen agents that execute a predetermined set of rules; these can be viewed as common trading strategies a trader might consider on a day-to-day basis. We call these agents "generators", though they can be viewed as "trained rules" because their hyperparameters are tuned with a genetic algorithm (see Appendix IV for the specific hyperparameter details and Appendix V for the genetic algorithm details). Many of these generators are, at least partially, based on popular trading strategies. The generators, in alphabetical order, are the following: bar movement, beep boop, Bollinger bands, choc, Keltner channels, MA crossover, MACD, MACD key level, MACD stochastic, PSAR, RSI, squeeze pro, stochastic, and supertrend.

Bar movement is a generator that checks the previous L price candles to see if price moved significantly in one particular direction. The rules for the strategy are given in Algorithm 1.

Algorithm 1: Bar movement strategy.

Calculate the previous L ATR [1] values. Calculate the average of these values, $\overline{A}_{TR}$, and multiply the average by a constant multiplier $P_{M_{ATR_M}}$ to obtain an adjusted ATR average $\tilde{\overline{A}}_{TR}$. Observe the most recent ATR value A_{TR}.

Observe the most recent middle closing price M_C (the "middle" price is halfway between the bid and ask prices). Round M_C to the nearest 100 pips; this is the nearest key level K_L.

Calculate whether M_C is near K_L. This is a boolean value N_{KL} and is calculated as follows: $N_{KL} = |M_C - K_L| < A_{TR} * N_{KL_M}$, where N_{KL_M} is a constant multiplier.

Calculate the most recent value of a specified price moving average M_A. Let the boolean value C_A be true if M_C is larger than the recent moving average value (price closed above the moving average), and false otherwise. If no moving average is specified, let C_A be true. Let the boolean value C_B be true if M_C is less than the recent moving average (price closed below the moving average), and false otherwise. If no moving average is specified, let C_B be true.

Use two variables to keep track of the bullish pips B_P and bearish pips B_{eP} of the previous L candles; initialize both to 0.

for *Each candle in the L most recent candles, in reverse order* **do**

  if *The candle is bullish* **then**

    Increment B_P by the candle body size ($|midopen - midclose|$).

  else

    Increment B_{eP} by the candle body size.

Let B_M be a boolean value that represents whether there was sufficient bullish movement, calculated as $B_M = B_P \geq \tilde{\overline{A}}_{TR}$. Let B_{eM} be a boolean value that represents whether there was sufficient bearish movement, calculated as $B_{eM} = B_{eP} \geq \tilde{\overline{A}}_{TR}$.

if N_{KL} *and* C_A *and* B_M **then**

  Place a buy trade.

else if N_{KL} *and* C_B *and* B_{eM} **then**

  Place a sell trade.

else

  Do not place a trade.

Beep boop (a sophisticated name, to be sure) is a generator that checks the previous N candles to see if MACD [2] movement was consistent in one particular direction. The rules for the strategy are given in Algorithm 2.

Algorithm 2: Beep boop strategy.

Calculate the previous N beep boop values. The beep boop technical indicator is derived from the MACD histogram value [2], the EMA50 value [3], the middle low price, and the middle high price. Its value is 1 if the MACD histogram is greater than 0 and the middle low price is larger than the EMA50 value. Its value is 2 if the MACD histogram is less than 0 and the middle high price is less than the EMA50 value. If the beep boop value is neither 1 nor 2, its value is 0.
Let B_B be a boolean value that represents whether the previous N beep boop values were all 1 (bullish). Let B_{eB} be a boolean value that represents whether the previous N beep boop values were all 2 (bearish).
Observe the most recent middle closing price M_C.
Calculate the most recent value of a specified moving average M_A. Let the boolean value C_A be true if M_C is larger than the recent moving average value (price closed above the moving average), and false otherwise. If no moving average is specified, let C_A be true. Let the boolean value C_B be true if M_C is less than the recent moving average (price closed below the moving average), and false otherwise. If no moving average is specified, let C_B be true.
if B_B *and* C_A **then**
    Place a buy trade.
else if B_{eB} *and* C_B **then**
    Place a sell trade.
else
    Do not place a trade.

Bollinger bands is a generator that uses the Bollinger bands indicator [4] when determining the best times to place trades. The rules for the strategy are given in Algorithm 3.

Algorithm 3: Bollinger bands strategy.

Observe the most recent middle closing price M_C.
Calculate the previous lower and upper Bollinger band values, L_{BB} and U_{BB}.
Let B_{BB} be a boolean value that represents whether M_C is less than L_{BB} (price closed below the lower band, which creates a bullish signal). Let B_{eBB} be a boolean value that represents whether M_C is larger than U_{BB} (price closed above the upper band, which creates a bearish signal).
Calculate the most recent value of a specified moving average M_A. Let the boolean value C_A be true if M_C is larger than the recent moving average value (price closed above the moving average), and false otherwise. If no moving average is specified, let C_A be true. Let the boolean value C_B be true if M_C is less than the recent moving average (price closed below the moving average), and false otherwise. If no moving average is specified, let C_B be true.
if B_{BB} *and* C_A **then**
    Place a buy trade.
else if B_{eBB} *and* C_B **then**
    Place a sell trade.
else
    Do not place a trade.

Choc ("change of character") is a generator that uses fractals [5] and observes whether price broke above resistance fractals or broke below support fractals. The rules for the strategy are given in Algorithm 4.

Algorithm 4: Choc strategy.

Calculate the most recent choc and ATR values, C_{HOC} and A_{TR}, respectively. If the closing price broke above a resistance fractal, C_{HOC} is 1. If the closing price broke below a support fractal, C_{HOC} is 2. If C_{HOC} is neither 1 nor 2, it is set to 0.
Let B_C be a boolean value that represents whether C_{HOC} is 1 (a bullish value). Let B_{eC} be a boolean value that represents whether C_{HOC} is 2 (a bearish value).
Observe the most recent middle closing price M_C. Round M_C to the nearest 100 pips; this is the nearest key level K_L.
Calculate whether M_C is near K_L. This is a boolean value N_{KL} and is calculated as follows: $N_{KL} = |M_C - K_L| < A_{TR} * N_{KL_M}$, where N_{KL_M} is a constant multiplier.
Calculate the most recent value of a specified moving average M_A. Let the boolean value C_A be true if M_C is larger than the recent moving average value (price closed above the moving average), and false otherwise. If no moving average is specified, let C_A be true. Let the boolean value C_B be true if M_C is less than the recent moving average (price closed below the moving average), and false otherwise. If no moving average is specified, let C_B be true.
if B_C *and* N_{KL} *and* C_A **then**
    Place a buy trade.
else if B_{eC} *and* N_{KL} *and* C_B **then**
    Place a sell trade.
else
    Do not place a trade.

Keltner channels is a generator that uses the Keltner channels indicator [6] when determining the best times to place trades. The rules for the strategy are given in Algorithm 5.

Algorithm 5: Keltner channels strategy.

Observe the most recent middle closing price M_C.
Calculate the previous lower and upper Keltner channel values, L_{KC} and U_{KC}.
Let B_{KC} be a boolean value that represents whether M_C is less than L_{KC} (price closed below the lower channel, which creates a bullish signal). Let B_{eKC} be a boolean value that represents whether M_C is larger than U_{KC} (price closed above the upper channel, which creates a bearish signal).
Calculate the most recent value of a specified moving average M_A. Let the boolean value C_A be true if M_C is larger than the recent moving average value (price closed above the moving average), and false otherwise. If no moving average is specified, let C_A be true. Let the boolean value C_B be true if M_C is less than the recent moving average (price closed below the moving average), and false otherwise. If no moving average is specified, let C_B be true.
if B_{KC} *and* C_A **then**
    Place a buy trade.
else if B_{eKC} *and* C_B **then**
    Place a sell trade.
else
    Do not place a trade.

MA ("moving average") crossover is a generator that uses two different price moving averages when determining whether to place a trade. The rules for the strategy are given in Algorithm 6.

Algorithm 6: MA crossover strategy.

Calculate the most recent values of two moving averages, M_{AS} and M_{AF}, where M_{AS} is a slow moving average and M_{AF} is a fast moving average (at least, faster than M_{AS}).
Let B_C be a boolean value that represents whether M_{AF} crossed above M_{AS} (a bullish crossover). Let B_{eC} be a boolean value that represents whether M_{AF} crossed below M_{AS} (a bearish crossover).
if B_C **then**
    Place a buy trade.
else if B_{eC} **then**
    Place a sell trade.
else
    Do not place a trade.

MACD is a generator that uses the MACD indicator [2] when determining whether to place a trade. The rules for the strategy are given in Algorithm 7.

Algorithm 7: MACD strategy.

Calculate the most recent MACD and MACD signal values, M_{ACD} and M_{ACD_S}. Note that different versions of the MACD indicator can be used.
Let B_C be a boolean value that represents whether M_{ACD} crossed above M_{ACD_S} (a bullish crossover). Let B_{eC} be a boolean value that represents whether M_{ACD} crossed below M_{ACD_S} (a bearish crossover).
Calculate the most recent ATR value and multiply it by a constant to obtain $\tilde{A}_{TR}$.
Let L_E be a boolean value that represents if the MACD values are large enough; it is normally calculated as $L_E = \min(M_{ACD}, M_{ACD_S}) \geq \tilde{A}_{TR}$. However, if the normalized MACD indicator [7] is being used, it is calculated as $L_E = \min(M_{ACD}, M_{ACD_S}) \geq M_{ACD_{Th}}$, where $M_{ACD_{Th}}$ is a MACD threshold parameter.
Observe the most recent middle closing price M_C.
Calculate the most recent value of a specified moving average M_A. Let the boolean value C_A be true if M_C is larger than the recent moving average value (price closed above the moving average), and false otherwise. If no moving average is specified, let C_A be true. Let the boolean value C_B be true if M_C is less than the recent moving average (price closed below the moving average), and false otherwise. If no moving average is specified, let C_B be true.
if B_C *and* $\max(M_{ACD}, M_{ACD_S}) < 0$ *and* L_E *and* C_A **then**
    Place a buy trade.
else if B_{eC} *and* $\min(M_{ACD}, M_{ACD_S}) > 0$ *and* L_E *and* C_B **then**
    Place a sell trade.
else
    Do not place a trade.

MACD key level is similar to the MACD generator, but checks if price is a near a key level before placing a trade. The rules for the strategy are given in Algorithm 8.

Algorithm 8: MACD key level strategy.

Calculate the most recent MACD and MACD signal values, M_{ACD} and M_{ACD_S}. Note that different versions of the MACD indicator can be used.
Let B_C be a boolean value that represents whether M_{ACD} crossed above M_{ACD_S} (a bullish crossover). Let B_{eC} be a boolean value that represents whether M_{ACD} crossed below M_{ACD_S} (a bearish crossover).
Calculate the most recent ATR value A_{TR} and multiply it by a constant to obtain $\tilde{A}_{TR}$.
Let L_E be a boolean value that represents if the MACD values are large enough; it is normally calculated as $L_E = \min(M_{ACD}, M_{ACD_S}) \geq \tilde{A}_{TR}$. However, if the normalized MACD indicator is being used, it is calculated as $L_E = \min(M_{ACD}, M_{ACD_S}) \geq M_{ACD_{Th}}$, where $M_{ACD_{Th}}$ is a MACD threshold parameter.
Observe the most recent middle closing price M_C. Round M_C to the nearest 100 pips; this is the nearest key level K_L.
Calculate whether M_C is near K_L. This is a boolean value N_{KL} and is calculated as follows: $N_{KL} = |M_C - K_L| < A_{TR} * N_{KL_M}$, where N_{KL_M} is a constant multiplier.
Calculate the most recent value of a specified moving average M_A. Let the boolean value C_A be true if M_C is larger than the recent moving average value (price closed above the moving average), and false otherwise. If no moving average is specified, let C_A be true. Let the boolean value C_B be true if M_C is less than the recent moving average (price closed below the moving average), and false otherwise. If no moving average is specified, let C_B be true.
if B_C *and* $\max(M_{ACD}, M_{ACD_S}) < 0$ *and* L_E *and* C_A *and* N_{KL} **then**
    Place a buy trade.
else if B_{eC} *and* $\min(M_{ACD}, M_{ACD_S}) > 0$ *and* L_E *and* C_B *and* N_{KL} **then**
    Place a sell trade.
else
    Do not place a trade.

MACD stochastic is also similar to the MACD generator, but checks for a stochastic indicator [8] cross before placing a trade. The rules for the strategy are given in Algorithm 9.

Algorithm 9: MACD stochastic strategy.

Calculate the most recent MACD and MACD signal values, M_{ACD} and M_{ACD_S}. Note that different versions of the MACD indicator can be used.
Let B_C be a boolean value that represents whether M_{ACD} crossed above M_{ACD_S} (a bullish crossover). Let B_{eC} be a boolean value that represents whether M_{ACD} crossed below M_{ACD_S} (a bearish crossover).
Calculate the most recent ATR value and multiply it by a constant to obtain $\tilde{A}_{TR}$.
Let L_E be a boolean value that represents if the MACD values are large enough; it is normally calculated as $L_E = \min(M_{ACD}, M_{ACD_S}) \geq \tilde{A}_{TR}$. However, if the normalized MACD indicator is being used, it is calculated as $L_E = \min(M_{ACD}, M_{ACD_S}) \geq M_{ACD_{Th}}$, where $M_{ACD_{Th}}$ is a MACD threshold parameter.
Observe the most recent middle closing price M_C.
Calculate the most recent value of a specified moving average M_A. Let the boolean value C_A be true if M_C is larger than the recent moving average value (price closed above the moving average), and false otherwise. If no moving average is specified, let C_A be true. Let the boolean value C_B be true if M_C is less than the recent moving average (price closed below the moving average), and false otherwise. If no moving average is specified, let C_B be true.
Calculate the previous L slow k and slow d values (using the stochastic indicator). Let S_{BC} be a boolean value that represents whether the slow k crossed above the slow d at any point during the previous L candles (a bullish stochastic cross) and both the slow k and slow d were less than 20. Let S_{BeC} be a boolean value that represents whether the slow k crossed below the slow d at any point during the previous L candles (a bearish stochastic cross) and both the slow k and slow d were larger than 80.
if B_C *and* $\max(M_{ACD}, M_{ACD_S}) < 0$ *and* L_E *and* C_A *and* S_{BC} **then**
    Place a buy trade.
else if B_{eC} *and* $\min(M_{ACD}, M_{ACD_S}) > 0$ *and* L_E *and* C_B *and* S_{BeC} **then**
    Place a sell trade.
else
    Do not place a trade.

PSAR is a generator that uses the parabolic SAR indicator [9] when determining whether to place a trade. The rules for the strategy are given in Algorithm 10.

Algorithm 10: PSAR strategy.

Calculate the most recent PSAR value, P_{SAR}.
Observe the most recent middle low, high, and closing prices, M_L, M_H, and M_C.
Let B be a boolean value that represents whether P_{SAR} was previously above M_H and then transitioned to being below M_L (a bullish PSAR signal). Let B_e be a boolean value that represents whether P_{SAR} was previously below M_L and then transitioned to being above M_H (a bearish PSAR signal).
Calculate the most recent value of a specified moving average M_A. Let the boolean value C_A be true if M_C is larger than the recent moving average value (price closed above the moving average), and false otherwise. If no moving average is specified, let C_A be true. Let the boolean value C_B be true if M_C is less than the recent moving average (price closed below the moving average), and false otherwise. If no moving average is specified, let C_B be true.
if B *and* C_A **then**
    Place a buy trade.
else if B_e *and* C_B **then**
    Place a sell trade.
else
    Do not place a trade.

RSI is a generator that uses the RSI (relative strength index) indicator [10] when deciding when to place trades. The rules for the strategy are given in Algorithm 11.

Algorithm 11: RSI strategy.

Calculate the most recent RSI value. Let B be a boolean value that represents whether the RSI value is less than 20. Let B_e be a boolean value that represents whether the RSI value is larger than 80.
Observe the most recent middle closing price M_C.
Calculate the most recent value of a specified moving average M_A. Let the boolean value C_A be true if M_C is larger than the recent moving average value (price closed above the moving average), and false otherwise. If no moving average is specified, let C_A be true. Let the boolean value C_B be true if M_C is less than the recent moving average (price closed below the moving average), and false otherwise. If no moving average is specified, let C_B be true.
if B *and* C_A **then**
    Place a buy trade.
else if B_e *and* C_B **then**
    Place a sell trade.
else
    Do not place a trade.

Squeeze pro is a generator that uses the TTM squeeze pro indicator [11] when deciding whether to place a trade. The rules for the strategy are given in Algorithm 12.

Algorithm 12: Squeeze pro strategy.

Calculate the most recent squeeze pro value and momentum color. Let B be a boolean value that represents whether the squeeze pro value changed from green to black, the momentum color is red or yellow, and there was a recent squeeze (a bullish squeeze signal). Let B_e be a boolean value that represents whether the squeeze pro value changed from green to black, the momentum color is blue or aqua, and there was a recent squeeze (a bearish squeeze signal). To determine if there was a "recent squeeze", calculate the previous L squeeze pro values. If there were R red squeeze values in a row, there was a recent squeeze; otherwise, there was no recent squeeze.
Observe the most recent middle closing price M_C.
Calculate the most recent value of a specified moving average M_A. Let the boolean value C_A be true if M_C is larger than the recent moving average value (price closed above the moving average), and false otherwise. If no moving average is specified, let C_A be true. Let the boolean value C_B be true if M_C is less than the recent moving average (price closed below the moving average), and false otherwise. If no moving average is specified, let C_B be true.
if B *and* C_A **then**
    Place a buy trade.
else if B_e *and* C_B **then**
    Place a sell trade.
else
    Do not place a trade.

Stochastic is a generator that uses the stochastic indicator [8] to check for a cross before placing a trade. The rules for the strategy are given in Algorithm 13.

Algorithm 13: Stochastic strategy.

Observe the most recent middle closing price M_C.
Calculate the most recent value of a specified moving average M_A. Let the boolean value C_A be true if M_C is larger than the recent moving average value (price closed above the moving average), and false otherwise. If no moving average is specified, let C_A be true. Let the boolean value C_B be true if M_C is less than the recent moving average (price closed below the moving average), and false otherwise. If no moving average is specified, let C_B be true.
Calculate the most recent slow k and slow d values (using the stochastic indicator). Let S_{BC} be a boolean value that represents whether the slow k crossed above the slow d (a bullish stochastic cross) and both the slow k and slow d were less than 20. Let S_{BeC} be a boolean value that represents whether the slow k crossed below the slow d (a bearish stochastic cross) and both the slow k and slow d were larger than 80.
if C_A *and* S_{BC} **then**
    Place a buy trade.
else if C_B *and* S_{BeC} **then**
    Place a sell trade.
else
    Do not place a trade.

Supertrend is a generator that uses the supertrend indicator [8] and QQE (qualitative quantitative estimation) mod indicator [12] when deciding to place trades. The rules for the strategy are given in Algorithm 14.

Algorithm 14: Supertrend strategy.

Calculate the most recent supertrend and QQE mod values. Let B be a boolean value that represents whether both the supertrend and QQE mod indicators give bullish signals. Let B_e be a boolean value that represents whether both the supertrend and QQE mod indicators give bearish signals. Note that the strategy may decide to ignore the QQE mod indicator (a hyperparameter controls this).
Observe the most recent middle closing price M_C.
Calculate the most recent value of a specified moving average M_A. Let the boolean value C_A be true if M_C is larger than the recent moving average value (price closed above the moving average), and false otherwise. If no moving average is specified, let C_A be true. Let the boolean value C_B be true if M_C is less than the recent moving average (price closed below the moving average), and false otherwise. If no moving average is specified, let C_B be true.
if B *and* C_A **then**
    Place a buy trade.
else if B_e *and* C_B **then**
    Place a sell trade.
else
    Do not place a trade.

III. Strategy for Price Forecasters

We used six different machine learning algorithms that were designed for price prediction: LSTM-Mixture, CNN with Grammian Angular Fields, KNN, LSTM, MLP, and Random Forest. Generating price predictions is one thing, but knowing how to use them to place trades is another. There are numerous ways to approach this problem. While not perfect, we feel our solution is reasonable, and is described in Algorithm 15. In summary, we predict where the bid and ask prices will move within the next candle. A trade is placed if the predictions indicate sufficient movement in a given direction.

Algorithm 15: Strategy/algorithm to convert price predictions to trades.

In reference to the current opening price, predict how many pips away the bid high, bid low, ask high, and ask low prices will be within the next candle. Let $\hat{b}_h$, $\hat{b}_l$, $\hat{a}_h$, and $\hat{a}_l$ represent the pips predictions, respectively.

Keep track of the previous 10 prediction errors for each of the four predictions. Specifically, let $E_{\hat{b}_h}$, $E_{\hat{b}_l}$, $E_{\hat{a}_h}$, and $E_{\hat{a}_l}$ represent the 10 most recent errors for $\hat{b}_h$, $\hat{b}_l$, $\hat{a}_h$, and $\hat{a}_l$, respectively.

Calculate the average values of $E_{\hat{b}_h}$, $E_{\hat{b}_l}$, $E_{\hat{a}_h}$, and $E_{\hat{a}_l}$. Let $\overline{E}_{\hat{b}_h}$, $\overline{E}_{\hat{b}_l}$, $\overline{E}_{\hat{a}_h}$, and $\overline{E}_{\hat{a}_l}$ represent the averages, respectively.

Multiply $\overline{E}_{\hat{b}_h}$, $\overline{E}_{\hat{b}_l}$, $\overline{E}_{\hat{a}_h}$, and $\overline{E}_{\hat{a}_l}$ by an error multiplier E_M. This is a hyperparameter described in Section IV with a default value of 0. Let the modified averages be $\tilde{\overline{E}}_{\hat{b}_h}$, $\tilde{\overline{E}}_{\hat{b}_l}$, $\tilde{\overline{E}}_{\hat{a}_h}$, and $\tilde{\overline{E}}_{\hat{a}_l}$, respectively.

To each of the four prediction values ($\hat{b}_h$, $\hat{b}_l$, $\hat{a}_h$, and $\hat{a}_l$), add the corresponding modified error average (i.e. $\hat{b}_h = \hat{b}_h + \tilde{\overline{E}}_{\hat{b}_h}$, $\hat{b}_l = \hat{b}_l + \tilde{\overline{E}}_{\hat{b}_l}$, etc.).

Observe the most recent middle closing price M_C.

Calculate the most recent value of a specified moving average M_A. Let the boolean value C_A be true if M_C is larger than the recent moving average value (price closed above the moving average), and false otherwise. If no moving average is specified, let C_A be true. Let the boolean value C_B be true if M_C is less than the recent moving average (price closed below the moving average), and false otherwise. If no moving average is specified, let C_B be true.

if $\max(\hat{b}_h, \hat{b}_l, \hat{a}_h, \hat{a}_l) = \hat{b}_h$ *and* C_A **then**

  Place a buy trade.

else if $\max(\hat{b}_h, \hat{b}_l, \hat{a}_h, \hat{a}_l) = \hat{a}_l$ *and* C_B **then**

  Place a sell trade.

else

  Do not place a trade.

After the current candle closes, observe the true pip values b_h, b_l, a_h, and a_l. Calculate the prediction errors and append them to $E_{\hat{b}_h}$, $E_{\hat{b}_l}$, $E_{\hat{a}_h}$, and $E_{\hat{a}_l}$, respectively. Since we only keep track of the 10 most recent errors, we also remove the errors at the beginning of the lists (the errors at index 0). There are numerous ways to achieve this functionality, but we used a deque collection [13].

IV. Hyperparameters

This section describes the various hyperparameters and their possible values for each algorithm/strategy we implemented. If the value of a hyperparameter is "none", it is ignored (i.e. an algorithm that uses the hyperparameter will not consider its value when determining when/how to place trades). The first subsection describes common hyperparameters that are shared between several strategies, mainly relating to risk management.

A. Shared Hyperparameters

The hyperparameters in this subsection are used by most of the algorithms, mainly to determine risk management procedures and identify market trends.

M_A is a parameter representing a particular price moving average (used to identify market trend). M_A can be one of the following values: none, EMA200, EMA100, EMA50, SMMA200, SMMA100, SMMA50. EMA is an exponential moving average, and SMMA [14] is a smoothed moving average. The numbers represent the moving average window. For example, EMA200 is an exponential moving average with a window of 200.

I is a boolean parameter representing whether to invert a trade; I can be true or false. For our experiments, inversion means to place a trade in the opposite direction, if a trade is signalled (if a trade is not signalled, then do nothing). For example, if an algorithm decides to place a buy and I is true, the algorithm will place a sell instead. Similarly, if an algorithm decides to place a sell and I is true, the algorithm will place a buy. If the reader is familiar with the popular sitcom *Seinfeld*, they can think of this as the "George Costanza" parameter [15].

T_{SL} is a boolean parameter representing whether to use a trailing stop-loss; T_{SL} can be true or false. If T_{SL} is true, a trade's stop-loss will be moved if the market moved in the trade's favor, and the stop-loss movement will be equal to the market movement. For example, if the trade is a buy and the market moved 20 pips in the trade's favor (up), the stop-loss will be moved up 20 pips. In contrast, if the market moved, say, 10 pips against the trade (down), the stop-loss would not be moved.

P_R is a parameter representing how many pips to initially risk (how many pips away from the open price the stop-loss should be placed). P_R can be one of the following values: none, 20, 30, 50, 100. If P_R is none, an algorithm will typically use the ATR bands indicator when choosing where to place the stop-loss (described later in Algorithm 16).

$P_{R_{ATR_M}}$ is a pips to risk ATR multiplier parameter. Its possible values are the following: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10. If an algorithm uses the ATR bands indicator when choosing where to place the stop-loss (instead of P_R, described in the previous

paragraph), it will multiply the number of pips to risk calculated from the indicator by $P_{R_{ATR_M}}$ (also described in Algorithm 16).

R_R is a parameter representing the reward-to-risk ratio. R_R can be one of the following values: none, 0.5, 1, 1.5, 2, 3, 4, 5. If R_R is none, the algorithm will not place a stop-gain level (instead, it will use a trailing stop-loss). Otherwise, the algorithm will use R_R to calculate where to place the stop-gain. For example, if R_R is 2 and the stop-loss is set 50 pips away from the open price, the stop-gain will be set $2 * 50 = 100$ pips away from the open price (in the direction of the trade).

With the common hyperparameters defined, we can now describe the default risk management approach via Algorithm 16. The majority of the individual strategies use this approach; if a particular strategy does not, we will make clarifications in its respective subsection.

Algorithm 16: Default risk management procedure.

Use the ATR bands indicator [16] to calculate the lower and upper ATR band values, L_{ATR} and U_{ATR}.
Calculate the number of pips to risk P_{TR}, where O_P represents the trade's open price (ask open for a buy, bid open for a sell):
if P_R *is not none* **then**
    $P_{TR} = P_R$.
else if *trade is a buy* **then**
    $P_{TR} = |O_P - L_{ATR}| * P_{R_{ATR_M}}$.
else
    $P_{TR} = |O_P - U_{ATR}| * P_{R_{ATR_M}}$.
Set the stop-loss P_{TR} pips away from the opening price O_P. If the trade is a buy, the stop-loss is placed below O_P and vice versa for a sell. Additionally, set the stop-loss as a trailing stop-loss if T_{SL} is true.
Calculate the number of pips to gain P_{TG}:
if R_R *is none* **then**
    $P_{TG} = 0$.
else
    $P_{TG} = R_R * P_{TR}$.
Set the stop-gain P_{TG} pips away from the open price O_P. If the trade is a buy, the stop-gain is placed above O_P and vice versa for a sell. If P_{TG} is 0 (i.e. R_R is none), then do not place a stop-gain; instead, be sure that the stop-loss is set as a trailing stop-loss.
Calculate the current spread $S = |A_O - B_O|$, where A_O is the current ask open price and B_O is the current bid open price.
if $S > P_{TR} * 0.1$ **then**
    Do not place the trade (because the spread is too large).
else
    Place the trade.

B. AlegAATr

AlegAATr is a bandit algorithm. As described in the main paper, the set of arms it can choose from is Γ, the set composed of the fourteen generators.

The only parameter AlegAATr uses is λ, which represents the probability of using empirical averages instead of AAT predictions. Specifically, with probability λ^{n_i}, where n_i is the number of time steps since AlegAATr selected generator $G_i \in \Gamma$, AlegAATr uses G_i's average empirical reward as its predicted value, instead of AAT. By default, AlegAATr keeps track of up to the previous 5 rewards for each generator; we did not change this value in our experiments. λ can be one of the following values: 0.9, 0.95, 0.99, 0.995, 1. $\lambda = 1$ was consistently the best value (i.e. avoid AAT predictions).

C. Bar Movement

Bar movement uses the default risk management procedure described in Algorithm 16; accordingly, it uses the T_{SL}, P_R, $P_{R_{ATR_M}}$, and R_R parameters. It also uses the M_A parameter to identify trends and the I parameter to (potentially) invert trades.

This strategy also uses N_{KL_M}, a multiplier to check how near the recent mid close price is to the nearest key level. It can be one of the following values: 0.5, 1, 1.5, 2, 2.5, 3.

Additionally, bar movement uses a pip movement ATR multiplier, $P_{M_{ATR_M}}$, that is used to adjust the average ATR value.

Finally, bar movement uses a lookback parameter L, that can be one of the following values: 6, 12, 24. This value is used to calculate the average ATR value as well as check previous candles for significant movement.

D. Beep Boop

Beep boop uses the default risk management procedure described in Algorithm 16; accordingly, it uses the T_{SL}, P_R, $P_{R_{ATR_M}}$, and R_R parameters. It also uses the M_A parameter to identify trends and the I parameter to (potentially) invert trades.

The only other parameter that beep boop uses is N, which represents the number of beep boop indicator values in a row must occur in order to place a trade. N can be one of the following values: 3, 5, 7, 9, 11, 13, 15.

E. Bollinger Bands

Bollinger bands uses the default risk management procedure described in Algorithm 16; accordingly, it uses the T_{SL}, P_R, $P_{R_{ATR_M}}$, and R_R parameters. It also uses the M_A parameter to identify trends and the I parameter to (potentially) invert trades. This strategy does not use any additional parameters.

F. Choc

Choc uses the default risk management procedure described in Algorithm 16; accordingly, it uses the T_{SL}, P_R, $P_{R_{ATR_M}}$, and R_R parameters. It also uses the M_A parameter to identify trends and the I parameter to (potentially) invert trades.

Similar to bar movement, choc also uses N_{KL_M}, a multiplier to check how near the recent mid close price is to the nearest key level. It can be one of the following values: 0.5, 1, 1.5, 2, 2.5, 3.

G. CNN

CNN uses the default risk management procedure described in Algorithm 16; accordingly, it uses the T_{SL}, P_R, $P_{R_{ATR_M}}$, and R_R parameters. It also uses the M_A parameter to identify trends and the I parameter to (potentially) invert trades.

As a price forecaster, CNN also uses the hyperparameter E_M, an error multiplier; see Algorithm 15 for more details. E_M can be one of the following values: 0, 0.5, 1, 1.5, 2, 2.5, 3.

H. EEE

EEE is a bandit algorithm. As described in the main paper, the set of arms it can choose from is Γ, the set composed of the fourteen generators.

The only parameter EEE uses is E_P, an exploration probability with one of the following values: 0.05, 0.1, 0.15, 0.2, 0.25. See [17] for more details.

I. Ensemble

As described in the main paper, ensemble attempts to combine the decisions of multiple generators. Specifically, its main parameter is a set of generators Γ_s, where $\Gamma_s \subseteq \Gamma$. Each $G_i \in \Gamma_s$ "votes" in regards to the type of trade and trade parameters it thinks should be used at any particular moment during a simulation. Γ_s can be any one of the members of the power set $\mathcal{P}(\Gamma)$; since there are numerous possible values (2^{14}), we do not explicitly list them here.

To help filter out noise, ensemble also uses a parameter M_{N_V}, which represents the minimum number of votes that must occur in order to place a trade. For example, if M_{N_V} is 3 and the majority vote is to place a buy, then there must be at least 3 buy votes (otherwise, no trade is made). M_{N_V} can be one of the following values: 1, 2, 3, 5, 7.

J. EXP4

EXP4 is a bandit algorithm. As described in the main paper, the set of arms it can choose from is Γ, the set composed of the fourteen generators.

The only parameter EXP4 uses is γ, a value used in arm weight calculations. γ can be one of the following values: 0.3, 0.4, 0.5, 0.6, 0.7, 0.8. See [18] for more details.

K. Keltner Channels

Keltner channels uses the default risk management procedure described in Algorithm 16; accordingly, it uses the T_{SL}, P_R, $P_{R_{ATR_M}}$, and R_R parameters. It also uses the M_A parameter to identify trends and the I parameter to (potentially) invert trades. Similar to Bollinger bands, this strategy does not use any additional parameters.

L. KNN

KNN uses the default risk management procedure described in Algorithm 16; accordingly, it uses the T_{SL}, P_R, $P_{R_{ATR_M}}$, and R_R parameters. It also uses the M_A parameter to identify trends and the I parameter to (potentially) invert trades.

As a price forecaster, KNN also uses the hyperparameter E_M, an error multiplier; see Algorithm 15 for more details. E_M can be one of the following values: 0, 0.5, 1, 1.5, 2, 2.5, 3.

M. LSTM

LSTM uses the default risk management procedure described in Algorithm 16; accordingly, it uses the T_{SL}, P_R, $P_{R_{ATR_M}}$, and R_R parameters. It also uses the M_A parameter to identify trends and the I parameter to (potentially) invert trades.

As a price forecaster, LSTM also uses the hyperparameter E_M, an error multiplier; see Algorithm 15 for more details. E_M can be one of the following values: 0, 0.5, 1, 1.5, 2, 2.5, 3.

N. LSTM-Mixture

LSTM-Mixture uses the default risk management procedure described in Algorithm 16; accordingly, it uses the T_{SL}, P_R, $P_{R_{ATR_M}}$, and R_R parameters. It also uses the M_A parameter to identify trends and the I parameter to (potentially) invert trades.

As a price forecaster, LSTM-Mixture also uses the hyperparameter E_M, an error multiplier; see Algorithm 15 for more details. E_M can be one of the following values: 0, 0.5, 1, 1.5, 2, 2.5, 3.

O. MA Crossover

MA crossover uses the default risk management procedure described in Algorithm 16; accordingly, it uses the T_{SL}, P_R, $P_{R_{ATR_M}}$, and R_R parameters. It also uses the I parameter to (potentially) invert trades, but does not use the M_A parameter to identify trends.

Instead of using the M_A parameter, MA crossover uses fast and slow moving averages, M_{AF} and M_{AS}, to identify trend changes. Both M_{AF} and M_{AS} can be one of the following values: EMA200, EMA100, EMA50, SMMA200, SMMA100, SMMA50. However, M_{AS}'s time period must be larger than M_{AF}'s time period (i.e. M_{AS} must be slower than M_{AF}).

P. MACD

MACD uses the default risk management procedure described in Algorithm 16; accordingly, it uses the T_{SL}, P_R, $P_{R_{ATR_M}}$, and R_R parameters. It also uses the M_A parameter to identify trends and the I parameter to (potentially) invert trades.

MACD uses another parameter M_{ACD_T}, which represents the type of MACD indicator to use. M_{ACD_T} can be either the regular MACD indicator [2], the normalized MACD indicator [7], or the impulse MACD indicator [19].

The final parameter that the MACD strategy uses is $M_{ACD_{Th}}$, a MACD threshold parameter that is used to either check the size of the normalized MACD value or, if using one of the other two MACD indicators, adjust the recent ATR value via multiplication to obtain $\tilde{A}_{TR}$ (see Algorithm 7 for more details). $M_{ACD_{Th}}$ can be one of the following values: 0, 0.5, 0.8, 0.95, 1, 1.5, 2.

Q. MACD Key Level

MACD key level uses the default risk management procedure described in Algorithm 16; accordingly, it uses the T_{SL}, P_R, $P_{R_{ATR_M}}$, and R_R parameters. It also uses the M_A parameter to identify trends and the I parameter to (potentially) invert trades.

Similar to MACD, MACD key level also uses M_{ACD_T}, a parameter that represents the type of MACD indicator to use. M_{ACD_T} can be either the regular MACD indicator, the normalized MACD indicator, or the impulse MACD indicator.

Also similar to MACD, MACD key level uses $M_{ACD_{Th}}$, a MACD threshold parameter that is used to either check the size of the normalized MACD value or, if using one of the other two MACD indicators, adjust the recent ATR value via multiplication to obtain $\tilde{A}_{TR}$. $M_{ACD_{Th}}$ can be one of the following values: 0, 0.5, 0.8, 0.95, 1, 1.5, 2.

Finally, MACD key level uses N_{KL_M} (similar to bar movement and choc), a multiplier to check how near the recent mid close price is to the nearest key level. It can be one of the following values: 0.5, 1, 1.5, 2, 2.5, 3.

R. MACD Stochastic

MACD stochastic uses the default risk management procedure described in Algorithm 16; accordingly, it uses the T_{SL}, P_R, $P_{R_{ATR_M}}$, and R_R parameters. It also uses the M_A parameter to identify trends and the I parameter to (potentially) invert trades.

Similar to MACD, MACD stochastic also uses M_{ACD_T}, a parameter that represents the type of MACD indicator to use. M_{ACD_T} can be either the regular MACD indicator, the normalized MACD indicator, or the impulse MACD indicator.

Also similar to MACD, MACD stochastic uses $M_{ACD_{Th}}$, a MACD threshold parameter that is used to either check the size of the normalized MACD value or, if using one of the other two MACD indicators, adjust the recent ATR value via multiplication to obtain $\tilde{A}_{TR}$. $M_{ACD_{Th}}$ can be one of the following values: 0, 0.5, 0.8, 0.95, 1, 1.5, 2.

MACD stochastic also uses a lookback parameter L, that can be one of the following values: 6, 12, 24. Similar to the bar movement strategy, this is a lookback parameter that is used to check for stochastic cross signals.

Finally, MACD stochastic uses a boolean parameter S_{RSI}, which represents whether the strategy should use RSI stochastic indicator [20] or the regular stochastic indicator [8].

S. MLP

MLP uses the default risk management procedure described in Algorithm 16; accordingly, it uses the T_{SL}, P_R, $P_{R_{ATR_M}}$, and R_R parameters. It also uses the M_A parameter to identify trends and the I parameter to (potentially) invert trades.

As a price forecaster, MLP also uses the hyperparameter E_M, an error multiplier; see Algorithm 15 for more details. E_M can be one of the following values: 0, 0.5, 1, 1.5, 2, 2.5, 3.

T. PSAR

PSAR uses most of the default risk management procedure described in Algorithm 16; specifically, it uses the T_{SL}, P_R, $P_{R_{ATR_M}}$, and R_R parameters. However, instead of using the ATR band values L_{ATR} and U_{ATR} when calculating P_{TR} (if P_R is none), $P_{TR} = |O_P - P_{SAR}| * P_{ATR_M}$. PSAR also uses the M_A parameter to identify trends and the I parameter to (potentially) invert trades.

U. Random Forest

Random forest uses the default risk management procedure described in Algorithm 16; accordingly, it uses the T_{SL}, P_R, $P_{R_{ATR_M}}$, and R_R parameters. It also uses the M_A parameter to identify trends and the I parameter to (potentially) invert trades.

As a price forecaster, random forest also uses the hyperparameter E_M, an error multiplier; see Algorithm 15 for more details. E_M can be one of the following values: 0, 0.5, 1, 1.5, 2, 2.5, 3.

V. RSI

RSI uses the default risk management procedure described in Algorithm 16; accordingly, it uses the T_{SL}, P_R, $P_{R_{ATR_M}}$, and R_R parameters. It also uses the M_A parameter to identify trends and the I parameter to (potentially) invert trades. Similar to Bollinger bands and Keltner channels, this strategy does not use any additional parameters.

W. Squeeze Pro

Squeeze pro uses the default risk management procedure described in Algorithm 16; accordingly, it uses the T_{SL}, P_R, $P_{R_{ATR_M}}$, and R_R parameters. It also uses the M_A parameter to identify trends and the I parameter to (potentially) invert trades.

Squeeze pro also uses a parameter R that represents the number of red squeeze values must occur in a row in order for a recent squeeze signal to occur. R can be one of the following values: 2, 3, 4, 5, 7, 9.

Finally, squeeze pro uses a lookback parameter L, that can be one of the following values: 25, 50, 100, 150, 200, 250, 300. Similar to the bar movement and MACD stochastic strategies, this is a lookback parameter that is used to check for recent squeeze signals.

X. Stochastic

Stochastic uses the default risk management procedure described in Algorithm 16; accordingly, it uses the T_{SL}, P_R, $P_{R_{ATR_M}}$, and R_R parameters. It also uses the M_A parameter to identify trends and the I parameter to (potentially) invert trades.

The only other parameter the stochastic strategy uses is a boolean parameter S_{RSI} (similar to the MACD stochastic strategy), which represents whether the strategy should use RSI stochastic indicator or the regular stochastic indicator.

Y. Supertrend

Similar to PSAR, supertrend uses most of the default risk management procedure described in Algorithm 16; specifically, it uses the T_{SL}, P_R, $P_{R_{ATR_M}}$, and R_R parameters. However, instead of using the ATR band values L_{ATR} and U_{ATR} when calculating P_{TR} (if P_R is none), $P_{TR} = (O_P - S_{LB}) * P_{ATR_M}$ for buys and $P_{TR} = (S_{UB} - O_P) * P_{ATR_M}$ for sells, where S_{LB} is the lower supertrend band and S_{UB} is the upper supertrend band. It also uses the M_A parameter to identify trends and the I parameter to (potentially) invert trades.

The only other parameter supertrend uses is a boolean parameter Q_{QE}, which represents whether the strategy should use the QQE mod indicator.

Z. Transformer

Transformer uses the default risk management procedure described in Algorithm 16; accordingly, it uses the T_{SL}, P_R, $P_{R_{ATR_M}}$, and R_R parameters. It also uses the M_A parameter to identify trends and the I parameter to (potentially) invert trades.

As a price forecaster, Transformer also uses the hyperparameter E_M, an error multiplier; see Algorithm 15 for more details. E_M can be one of the following values: 0, 0.5, 1, 1.5, 2, 2.5, 3.

. *UCB*

UCB is a bandit algorithm. As described in the main paper, the set of arms it can choose from is Γ, the set composed of the fourteen generators.

The only parameter UCB uses is δ, a value used in upper bound calculations. δ can be one of the following values: 0.9, 0.95, 0.99. See [21] for more details.

V. Genetic Algorithm

For each agent, currency pair, and time frame, we employed a genetic algorithm (GA) to tune the numerous hyperparameters described in Section IV. The GA runs 50 iterations, where each iteration contains a population of 20 genomes. The specific details are given in Algorithm 17. Note that genomes represent randomly-selected hyperparameter values pertaining to the strategy being tuned. Specifically, a genome's chromosome is the vector representation of its genetic information (the specific values of the hyperparameters). For example, RSI uses the T_{SL}, P_R, $P_{R_{ATR_M}}$, R_R, M_A, and I parameters. If an RSI genome were to be created (by randomly selecting values for each of these parameters), its chromosome would look something like the following: [true, 50, 6, none, SMMA50, false].

Note that 20 genomes might be view by some as a small population. Due to our limited computing resources and the size of the LSTM-Mixture model, 50 iterations and populations of 20 genomes were all we could run in a reasonable amount of time for LSTM-Mixture. To be fair to the other algorithms, we used these same GA parameters. However, we typically saw quick convergence in nearly every scenario, so we do not feel this is a cause for concern.

Algorithm 17: The genetic algorithm we used for tuning purposes in our experiments.

Initialize a population P of 20 genomes.
Initialize the best genome B_G to none.
for *i in the range 0 to 50* **do**
    Calculate the performance of each genome in the population. This is done by creating a version of the strategy being tuned with the specific hyperparameter values represented by the genome. The strategy is then run through a market trading simulation on a given year range, time frame, and currency pair. The genome's performance is the net profit achieved during the simulation.
    Pick the two genomes with the largest performances; create a list of genomes L_G that contains these two genomes. Let $|L_G|$ represent its length (i.e. $|L_G|$ is initially 2). Let M_P represent the net profit of the genome with the maximum/largest performance.
    if $M_P > 0$ **then**
        Set B_G equal to the genome with performance M_P.
    while $|L_G| < 20$ **do**
        Randomly select two genomes from the population P, where genomes are weighted by their performance (i.e. genomes with larger net profits are more likely to be selected). These two genomes can be viewed as parents.
        Create two offspring O_{S_1} and O_{S_2} by performing single point crossover on the two parents selected in the previous step. Single point crossover works by randomly selecting a point on both parents' chromosomes. Values to the right of that point are swapped between the two parent chromosomes, creating two children.
        Randomly mutate both O_{S_1} and O_{S_2} with a mutation ratio $M_R = 0.25$. Mutation works by randomly selecting chromosome values to change and then randomly changing those values. We set a maximum number of mutations that can occur to be $floor(N_P * M_R)$, where N_P is the number of parameters in the chromosome (i.e. the length of the chromosome).
        Add O_{S_1} and O_{S_2} to the list of genomes L_G.
    Update P by creating a new population composed of the 20 genomes in L_G.

VI. Bank Simulations

We implemented and ran a simple experiment designed to shed light on the nature of powerful agents within forex trading. This section will describe the specific details; in summary, we created a variety of agents, many of which are derived from the agents we studied during our two main experimentation phases. These agents are either deep Q-learners, bandits, or generators, where we label the former two as "learners". Similar to the agents described in Appendix II, the generators in this simulation represent a set of execution rules that can be viewed as strategies that an average trader might use. However, because this was a simple experiment, we did not run our genetic algorithm for tuning purposes. During each simulation, we used fifty of these agents to represent individual traders in the market. We also created a separate deep Q-learner that represents a large bank. The bank's gains are the losses of the individual traders, and vice versa.

A. Agents

This subsection describes the various agents we implemented and used for the bank simulations.

1) Bandits: Similar to our main two experimentation phases, we used EEE, EXP4, and UCB. However, we did not use AlegAATr due to limited time (one of AlegAATr's weaknesses is that it requires training for each of its generators). Also note that, instead of a bandit's arms being equal to the set of generators Γ, for this experiment the arms are the set of actions $\{0, 1, 2\}$, where 0 represents the "do nothing" action, 1 represents the "place a buy trade" action, and 2 represents the "place a sell trade" action. For EEE we let $E_P = 0.1$, for EXP4 we let $\gamma = 0.5$, and for UCB we let $\delta = 0.99$.

2) Deep Q-Learners: We used deep Q-learners as both regular agents as well as the bank agent. The reason we used a deep Q-learner for the bank is because we wanted it to be able to adapt to its environment as well as examine the current state of the environment.

Specifically, we used a discount factor d of 0.99 (for discounting rewards), an epsilon ϵ of 0.1 (for exploration), an epsilon decay ϵ_d of 0.99 (for decreasing exploration over time), a replay buffer of size 50000, a batch size of 512, an action dimension of 3 (actions can be either 0, 1, or 2, like the bandit agents), and a state dimension of 18. Note that, for the bank, the state dimension is 20 (described in the next paragraph). We used the Adam optimizer and a learning rate of 0.001.

A state passed to a deep Q-learner is represented as the vector composed of the most recent SMA50 (simple moving average with a window of size 50), SMA25, EMA50, EMA25, SMMA50, SMMA25, RSI, RSI simple moving average with a window of size 25, RSI slow k, RSI slow d, MACD, MACD signal, MACD historgram, QQE up, QQE down, QQE, market close, and account balance (the balance of the specific agent) values (18 values in total). For the bank agent, the state vector contains two additional items: the number of buy trades in the market and the number of sell trades. We feel this is realistic because large banks would likely have access to this information or at least have an estimate [22], [23]. If the reader does not agree with this, then please note that we also conducted experiments where the bank did not have this information, and we did not observe any significant differences.

With probability ϵ, a deep Q-learner chooses a random action. Otherwise, it predicts the Q-values of each action and selects the action with the largest value. After each simulation iteration, the Q-learner updates its network by randomly sampling 512 experiences from its replay buffer. After each episode, ϵ is updated as $\epsilon = \epsilon * \epsilon_d$.

3) Generators: We used simplified versions of the MACD, MACD stochastic, RSI, and stochastic generators; see Algorithms 18, 19, 20, and 21, respectively, for more details.

Algorithm 18: MACD strategy (bank simulation).

Calculate the most recent MACD and MACD signal values, M_{ACD} and M_{ACD_S}. Note that we only use the regular/default MACD indicator.
Let B_C be a boolean value that represents whether M_{ACD} crossed above M_{ACD_S} (a bullish crossover). Let B_{eC} be a boolean value that represents whether M_{ACD} crossed below M_{ACD_S} (a bearish crossover).
if B_C **then**
    Place a buy trade.
else if B_{eC} **then**
    Place a sell trade.
else
    Do not place a trade.

Algorithm 19: MACD stochastic strategy (bank simulation).

Calculate the most recent MACD and MACD signal values, M_{ACD} and M_{ACD_S}. Note that we only use the regular/default MACD indicator.
Let B_C be a boolean value that represents whether M_{ACD} crossed above M_{ACD_S} (a bullish crossover). Let B_{eC} be a boolean value that represents whether M_{ACD} crossed below M_{ACD_S} (a bearish crossover).
Calculate the previous 12 slow k and slow d values (using the stochastic indicator). Let S_{BC} be a boolean value that represents whether the slow k crossed above the slow d at any point during the previous 12 candles (a bullish stochastic cross) and both the slow k and slow d were less than 20. Let S_{BeC} be a boolean value that represents whether the slow k crossed below the slow d at any point during the previous 12 candles (a bearish stochastic cross) and both the slow k and slow d were larger than 80.
if B_C *and* S_{BC} **then**
    Place a buy trade.
else if B_{eC} *and* S_{BeC} **then**
    Place a sell trade.
else
    Do not place a trade.

Algorithm 20: RSI strategy (bank simulation).

Calculate the most recent RSI value. Let B be a boolean value that represents whether the RSI value is less than 20. Let B_e be a boolean value that represents whether the RSI value is larger than 80.
if B **then**
    Place a buy trade.
else if B_e **then**
    Place a sell trade.
else
    Do not place a trade.

Algorithm 21: Stochastic strategy (bank simulation).

Calculate the most recent slow k and slow d values (using the stochastic indicator). Let S_{BC} be a boolean value that represents whether the slow k crossed above the slow d (a bullish stochastic cross) and both the slow k and slow d were less than 20. Let S_{BeC} be a boolean value that represents whether the slow k crossed below the slow d (a bearish stochastic cross) and both the slow k and slow d were larger than 80.
if S_{BC} **then**
    Place a buy trade.
else if S_{BeC} **then**
    Place a sell trade.
else
    Do not place a trade.

4) Learners: When we refer to "learner" agents, we simply mean agents that are either bandits or deep Q-learners. This is because these agents change their strategy over time (i.e. they try to learn the best policy), as compared to the static generators.

B. Environment

During each simulation, we use 50 agents (that represent individual traders) plus one bank agent, for a total of 51 agents. Similar to our main experiments, each agent begins a simulation with an account balance of \$10,000 and risks 2% of its balance per trade. However, the bank begins with a \$500,000 account balance (50 agents, multiplied by \$10,000). This is designed to, at the beginning of every simulation, ensure that the market power of the bank is equal to the collective market power of the 50 agents.

As this is a simple simulation, we do not use bid and ask prices; we simply use a single price. At the beginning of each simulation, the initial price/exchange rate is set to 1. As trades are placed, the price changes to reflect market demand. Specifically, price P is update as $P = P + \sum_{t \in T} sign(t) * \frac{t_d * 0.0001}{|A|}$, where T is the set of new incoming trades, t is an individual trade (a member of T), $sign(t)$ is a function that returns 1 if t is a buy and -1 if t is a sell, t_d is the dollar amount

of the trade, $t_d * 0.0001$ represents a conversion from dollars to pips, and $|A|$ is the number of agents (we used a total of 51 agents in every simulation).

Note that, also similar to our main experiments, an agent can only place a single trade at a time. In order to avoid stagnation, we automatically close trades that have been open for 7 time periods. We also estimate "day" fees as 1% of a trade's dollar amount for every time step the trade is open.

Finally, the gains of individual agents are the losses of the bank, and vice versa. This gives the bank incentive to "attack" the positions of the individual agents (i.e. trade in the opposite direction).

C. Experiment Conditions

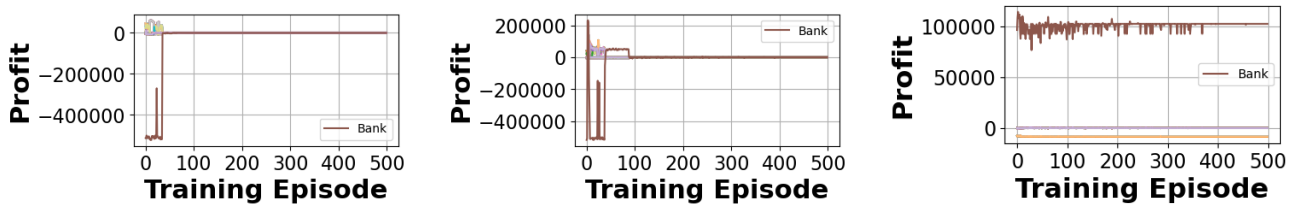
Every simulation runs until the bank agent runs out of money, all of the other 50 agents run out of money, or for a maximum of 100 time steps. Each experiment runs for a total of 1000 simulation episodes. As episodes are run, the learner agents gradually update their policies.

We conducted three experiments:

- Learners: In this experiment, the bank agent faced 12 UCB agents, 12 EEE agents, 12 EXP4 agents, and 14 deep Q-learner agents. In other words, the bank faced a variety of learner agents.
- Variety: In this experiment, the bank agent faced 6 UCB agents, 6 MACD agents, 6 EEE agents, 6 EXP4 agents, 6 deep Q-learner agents, 6 RSI agents, 6 MACD stochastic agents, and 8 stochastic agents. In other words, the bank faced a variety of every agent type.
- Generators: In this experiment, the bank agent faced 12 MACD agents, 12 RSI agents, 12 MACD stochastic agents, and 14 stochastic agents. In other words, the bank faced a variety of generators.

D. Results

Figure 1 shows results from three simulation conditions. Each subplot shows the profit of each agent over time as the bank and other learners attempt to discover optimal policies. Subplots (a) and (b) show results when at least some of the individual agents learn. Both the bank and the individual learners eventually decided that the best thing to do was, simply, nothing (i.e., avoid trading). In the second subplot, where individuals are either learners or trained rules, the profits of the trained rules also converged to $0 because there was no longer any market movement. In subplot (c), the bank very quickly learned to take advantage of the trained rules, as they do not adapt their strategy. These results illustrate that an astute bank will quickly change behavior when the market does not yield profits (first two cases). However, when individual traders are less sophisticated than the bank (third case, which, we argue, is common since banks have more resources than individuals), the bank takes profits from individual traders.



(a) Traders learn (b) Variety of traders (c) Traders do not learn

Fig. 1. Profits over time of a bank and individual traders in simple simulations. (a) Every individual agent is a bandit or deep Q-learner. (b) Individual traders are learning algorithms or trained rules. (c) Every individual agent is a trained rule (they do not learn). The bank is a deep Q-learner in every scenario.

VII. Supplementary Results

In this section, we present results that we did not include in the main paper.

A. LSTM-Mixture Additional Results

Aside from predicting a single candle into the future, we tested predictions for both three and five candles into the future for LSTM-Mixture (since it was the best price forecaster algorithm). In both cases, LSTM-Mixture's performance was nearly identical to its single-candle performance (average profit of approximately -$750). In the three-candle case, it rose one spot in the "rankings". It dropped six spots in the rankings in the five-candle case.

Group 1	Group 2	Mean Diff	P-adj
AlegAATr	EEE	-392.126	0.384
AlegAATr	EXP4	-222.368	0.803
AlegAATr	UCB	-189.437	0.868
EEE	EXP4	169.758	0.901
EEE	UCB	202.689	0.843
EXP4	UCB	32.931	0.999

TABLE I
STATISTICAL COMPARISONS ON PROFIT AMOUNTS BETWEEN BANDITS.

B. Statistical Tests

Table I shows the results of a multiple comparison test between the bandit algorithms. We can see that there is no statisitcal significance.

Table II shows the effect sizes when comparing AlegAATr's profit amounts to the profit amounts of every other algorithm. We can see that, in the majority of cases, the effect size is is either medium or large. However, one should take these results with a grain of salt, as there is not much statistical significance.

Comparison	Effect Size
AlegAATr vs. Transformer	0.461
AlegAATr vs. Supertrend	0.390
AlegAATr vs. MLP	0.830
AlegAATr vs. UCB	0.201
AlegAATr vs. CNN	0.521
AlegAATr vs. EXP4	0.172
AlegAATr vs. Keltner Channels	0.603
AlegAATr vs. Squeeze Pro	0.350
AlegAATr vs. Beep Boop	0.796
AlegAATr vs. MACD	0.238
AlegAATr vs. Choc	0.095
AlegAATr vs. LSTM	0.529
AlegAATr vs. EEE	0.247
AlegAATr vs. MA Crossover	0.477
AlegAATr vs. LSTM-Mixture	0.605
AlegAATr vs. RSI	0.144
AlegAATr vs. Random Forest	0.724
AlegAATr vs. Stochastic	0.522
AlegAATr vs. Bollinger Bands	0.524
AlegAATr vs. KNN	0.499
AlegAATr vs. PSAR	0.673
AlegAATr vs. Ensemble	0.518
AlegAATr vs. MACD Stochastic	0.445
AlegAATr vs. Bar Movement	0.471
AlegAATr vs. MACD Key Level	0.360

TABLE II
PROFIT AMOUNT EFFECT SIZES (COHEN'S D) BETWEEN ALEGAATR AND EVERY OTHER STRATEGY, ROUNDED TO THREE DECIMALS. AS A GENERAL RULE OF THUMB, VALUES LESS THAN 0.2 ARE CONSIDERED SMALL, VALUES AROUND 0.5 ARE CONSIDERED MEDIUM, AND VALUES LARGER THAN 0.8 ARE CONSIDERED LARGE.

C. Profit Distributions

Examining the profit distributions (from phase two) for each strategy can also help us understand the zero-sum nature of this domain, as illustrated in Figure 2. Rather than focus on average profit amount, this figure shows the distribution of profit amounts (box plots) across every test category from the second experimentation phase. We can see that, for nearly every strategy, 0 falls between the first and third quartiles.

D. Prediction Distributions

To further highlight the challenges of creating a model with low variance, we analyzed the distributions of predictions for a few of the algorithms we used. Specifically, we chose AlegAATr, UCB, and LSTM-Mixture. We selected AlegAATr and UCB because they were the best bandits, and we chose LSTM-Mixture because it was the best price forecaster. Figure 3 shows their predictions across every test scenario from the second experimentation phase, separated into two groups: "correct" predictions that resulted in profitable trades and "incorrect" predictions that resulted in negative trades. We can see that, for each algorithm, the distributions are roughly identical, indicating that, perhaps, there is so much noise in the market that generating accurate predictions is extremely difficult.

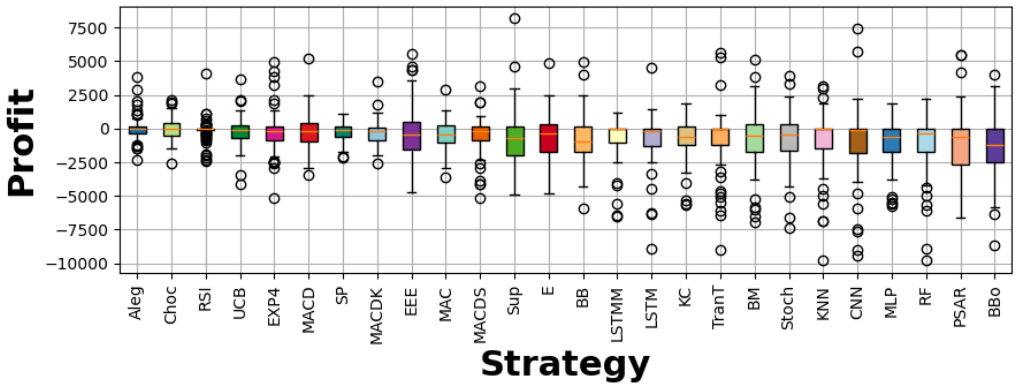


Fig. 2. Distributions of profit amounts over ten years of testing (from phase two).

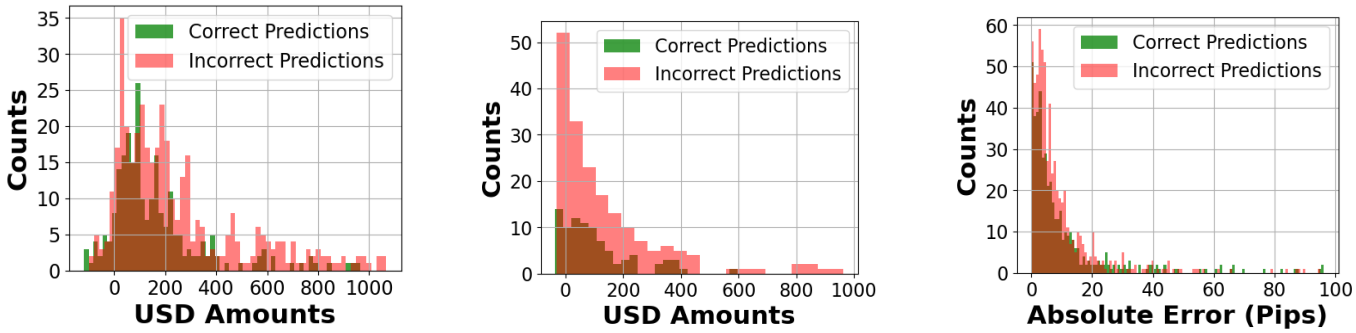


Fig. 3. Prediciton distributions for AlegAATr, UCB, and LSTM-Mixture, respectively, from phase two. Green values represent "correct" predictions that resulted in profitable trades and red values represent "incorrect" predictions that resulted in negative trades. Note that, because AlegAATr and UCB are bandits, their predictions are actual dollar amounts, whereas LSTM-Mixture predicts future prices (so we plotted prediction errors in the third subplot).

References

[1] A. Hayes, "Average true range (atr) formula, what it means, and how to use it," *Investopedia* – https://www.investopedia.com/terms/a/atr.asp, 2024, accessed: 2024-03-04.

[2] B. Dolan, "Macd indicator explained with formula, examples, and limitations," *Investopedia* – https://www.investopedia.com/terms/m/macd.asp, 2024, accessed: 2024-03-04.

[3] J. Chen, "What is ema? how to use exponential moving average with formula," *Investopedia* – https://www.investopedia.com/terms/e/ema.asp, 2024, accessed: 2024-03-04.

[4] C. Thompson, "Bollinger bands®: What they are, and what they tell investors," *Investopedia* – https://www.investopedia.com/terms/b/bollingerbands.asp, 2024, accessed: 2024-03-04.

[5] C. Mitchell, "Fractal indicator: Definition, what it signals, and how to trade," *Investopedia* – https://www.investopedia.com/terms/f/fractal.asp, 2022, accessed: 2024-03-04.

[6] ——, "Keltner channel: Definition, how it works, and how to use," *Investopedia* – https://www.investopedia.com/terms/k/keltnerchannel.asp, 2023, accessed: 2024-03-04.

[7] I. TradingView, "Normalized macd," *TradingView* – https://www.tradingview.com/script/d64lSwoT-Normalized-MACD/, 2015, accessed: 2024-03-13.

[8] A. Hayes, "Stochastic oscillator: What it is, how it works, how to calculate," *Investopedia* – https://www.investopedia.com/terms/s/stochasticoscillator.asp, 2023, accessed: 2024-03-05.

[9] C. Mitchell, "Parabolic sar indicator: Definition, formula, trading strategies," *Investopedia* – https://www.investopedia.com/terms/p/parabolicindicator.asp, 2024, accessed: 2024-03-05.

[10] J. Fernando, "Relative strength index (rsi) indicator explained with formula," *Investopedia* – https://www.investopedia.com/terms/r/rsi.asp, 2024, accessed: 2024-03-05.

[11] I. TradingView, "Ttm squeeze pro," *TradingView* – https://www.tradingview.com/script/0drMdHsO-TTM-Squeeze-Pro/, 2021, accessed: 2024-03-05.

[12] ——, "Qqe mod," *TradingView* – https://www.tradingview.com/script/TpUW4muw-QQE-MOD/, 2020, accessed: 2024-03-05.

[13] T. P. S. Foundation, "collections — container datatypes," *Python* – https://docs.python.org/3.9/library/collections.html#collections.deque, n.d., accessed: 2024-02-23.

[14] K. D. Turck, "The smoothed moving average (smma)," *ChartMill* – https://www.chartmill.com/documentation/technical-analysis/indicators/217-The-Smoothed-Moving-Average-SMMA, 2024, accessed: 2024-03-11.

[15] I. IMDb.com, "The opposite," *IMDb* – https://www.imdb.com/title/tt0697744/, n.d., accessed: 2024-03-11.

[16] I. TradingView, "Atr bands," *TradingView* – https://www.tradingview.com/script/ziTzsSfo-ATR-Bands/, 2022, accessed: 2024-03-11.

[17] D. P. De Farias and N. Megiddo, "Combining expert advice in reactive environments," *Journal of the ACM (JACM)*, vol. 53, no. 5, pp. 762–799, 2006.

[18] P. Auer, N. Cesa-Bianchi, Y. Freund, and R. E. Schapire, "The nonstochastic multiarmed bandit problem," *SIAM journal on computing*, vol. 32, no. 1, pp. 48–77, 2002.

[19] I. TradingView, "Impulse macd," *TradingView* – https://www.tradingview.com/script/qt6xLfLi-Impulse-MACD-LazyBear/, 2015, accessed: 2024-03-13.

[20] A. Hayes, "Stochastic rsi -stochrsi definition," *Investopedia* – https://www.investopedia.com/terms/s/stochrsi.asp, 2021, accessed: 2024-03-13.
[21] P. Auer, "Using confidence bounds for exploitation-exploration trade-offs," *Journal of Machine Learning Research*, vol. 3, no. Nov, pp. 397–422, 2002.
[22] L. Babypips.com, "How forex brokers manage their risk and make money," *babypips* – https://www.babypips.com/learn/forex/how-forex-brokers-manage-risk-make-money, n.d., accessed: 2024-03-15.
[23] ——, "A-book: How forex brokers manage their risk," *babypips* – https://www.babypips.com/learn/forex/a-book-forex-brokers, n.d., accessed: 2024-03-15.